Project 1

Project: Large-Scale Evaluation of Modern Search and Sorting Algorithms

1. Objective

Students will **study modern/state-of-the-art algorithms**, implement them, and evaluate their performance by generating **1,000,000 random numbers** and performing sorting and searching tasks.

The project focuses on:

- Algorithm analysis
- Experimental performance evaluation
- Large-scale data processing

Part 1 — State-of-the-Art Sorting Algorithms

Students should select 3 modern sorting algorithms such as AlphaDev Sorting Algorithm , PDQSort, Timsort or any other sort algorithm introduced recently 2020…

1. AlphaDev Sorting Algorithm

- AI-discovered algorithm from reinforcement learning.
- Found more efficient instruction-level sorting routines.
- Improved latency compared with human-designed algorithms.

Link:
https://www.nature.com/articles/s41586-023-06004-9

Concept:

- Reinforcement learning generates faster instruction sequences.

---

2. Pattern-Defeating Quicksort (PDQSort)

- Modern improvement over quicksort.

- Detects patterns in input data.
- Linear time for some input distributions.

Link:
https://github.com/orlp/pdqsort

Key idea:

- Combines quicksort + heapsort strategies.

Complexity:

- Average: **O(n log n)**
- Best case: **O(n)**

---

3. Timsort

Hybrid algorithm combining merge sort and insertion sort.

Used in:

- Python
- Java
- Android

It identifies already-sorted subsequences (“runs”) to reduce comparisons.

Complexity:

- Best: **O(n)**
- Average: **O(n log n)**

---

4. Recent Research Algorithms (Optional Advanced)

Students may explore recent research algorithms:

- **TwinArray Sort (2024)**
    - Non-comparison sort
    - Near linear time in many cases.

- **EvoSort (2025)**
    - Genetic algorithm optimized parallel sorting.
- **XiSort (2025)**
    - Deterministic sorting for floating-point data.

Part 2 — Modern Search Algorithms

Students implement traditional 3 search algorithms such as bubble sort, quick sort ….

Part 3 — Dataset Generation

Students generate 1 million numbers.

Variants to test:

- Random numbers
- Nearly sorted
- Reverse sorted
- Duplicate values

Part 4 — Experiment Tasks

Students must perform:

1. Sorting Experiment

Run: the selected algorithms such as PDQSort, Timsort, and QuickSort

Measure: Execution time, Memory usage and Number of comparisons

---

---

2. Search Experiment

After sorting:

Search **1000 random queries**

Algorithms: select 3 different algorithms such as

- Binary search, Exponential search, Linear search, M-Way tree (Select suitable M may be 10 ?) , and skipList

Measure: Average search time, Worst case time, memory usage.

Part 5 — Expected Output

Students must submit:

1. Source Code

Python / C++ / Java / C# …

2. Performance Graphs

Examples:

- Time vs dataset size
- Sorting algorithm comparison
- Search algorithm comparison

---

3. Report

Sections:

1. Introduction
2. Algorithms overview
3. Experimental setup
4. Results
5. Discussion
6. Conclusion

Bonus Ideas (for top students)

1. GPU sorting using CUDA
2. Parallel sorting with OpenMP
3. AI-based algorithm discovery (like AlphaDev and Learned Index search)
4. Sorting 100 million numbers
